

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS ARARANGUÁ
GRADUAÇÃO EM TECNOLOGIA DA INFORMAÇÃO E COMUNICAÇÃO

Marco Antonio Baltazar Vieira e Mikael Anastácio Teixeira

TRABALHO PRÁTICO UNIDADE II

Manutenção e Evolução de um Projeto Open Source

Araranguá
2026

1 INTRODUÇÃO..... 3
2 IDENTIFICAÇÃO DO PROJETO E JUSTIFICATIVA DA ESCOLHA..... 3
3 CARACTERIZAÇÃO E COMPREENSÃO INICIAL DO SISTEMA..... 3
4 BASELINE..... 4
5 PROBLEMAS DE MANUTENIBILIDADE IDENTIFICADOS.....5
6 REFATORAÇÕES REALIZADAS.....6
6.1 Rede de segurança de testes..... 6
6.2 Refatoração 1: movimento duplicado (P1).....6
6.3 Refatoração 2: colisão sem nome (P2).....7
6.4 Refatoração 3: velocidade como estado derivado (P3)..... 7
6.5 Preservação do comportamento..... 8
7 SOLICITAÇÃO DE MANUTENÇÃO PERFECTIVA..... 8
8 ANÁLISE DE IMPACTO..... 9
9 DESCRIÇÃO DA IMPLEMENTAÇÃO..... 10
10 TESTES E EVIDÊNCIAS DE REGRESSÃO..... 12
11 IMPACTO PREVISTO VERSUS OBSERVADO.....13
12 REFLEXÃO SOBRE MODIFICAR SOFTWARE DE TERCEIROS..... 14

1 INTRODUÇÃO

Este relatório descreve a manutenção de um software desenvolvido por terceiros, realizada no Trabalho Prático da Unidade II. A dupla escolheu um projeto *open source*, registrou uma *baseline*, analisou problemas de manutenibilidade, fez três refatorações e implementou uma manutenção perfectiva: **muros no campo de jogo**. O foco do texto são as análises, justificativas e decisões; o código antes e depois de cada mudança pode ser consultado nos *commits* indicados.

O repositório público da dupla é

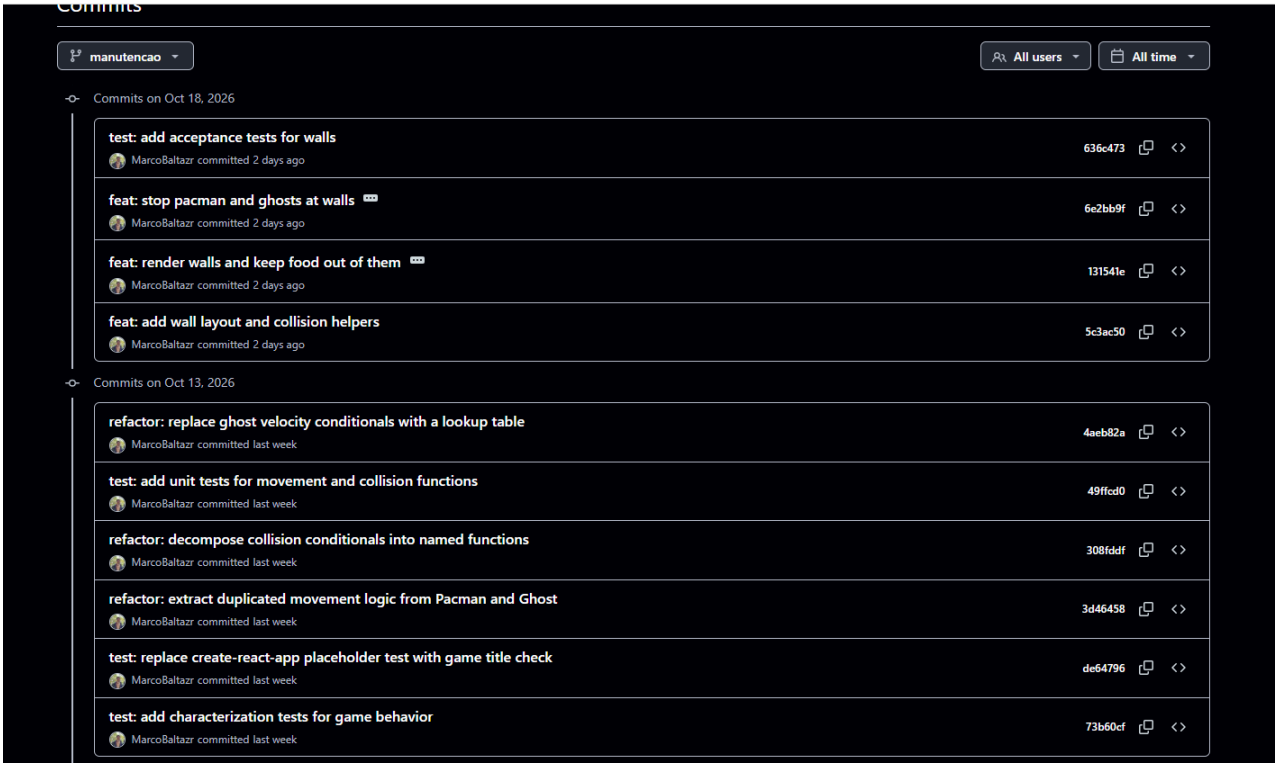
https://github.com/MarcoBaltazr/pacman-eng-software. A *branch* principal (master) contém a versão original do projeto, sem modificações, no *commit* da *baseline* (e17bd96). A *branch* manutencao contém as alterações da dupla, registradas nos 10 *commits* da Tabela 1.

Tabela 1 – Rastreabilidade: commits da branch manutencao

Commit	**Mensagem**	**Etapa**
73b60cf	test: add characterization tests for game behavior	Rede de segurança (seção 5.1)
de64796	test: replace create-react-app placeholder test with game title check	Rede de segurança (seção 5.1)
3d46458	refactor: extract duplicated movement logic from Pacman and Ghost	Refatoração 1 (P1)
308fddf	refactor: decompose collision conditionals into named functions	Refatoração 2 (P2)
49ffcd0	test: add unit tests for movement and collision functions	Testes das refatorações 1 e 2
4aeb82a	refactor: replace ghost velocity conditionals with a lookup table	Refatoração 3 (P3)
5c3ac50	feat: add wall layout and collision helpers	Manutenção perfectiva
131541e	feat: render walls and keep food out of them	Manutenção perfectiva (corrige P4)
6e2bb9f	feat: stop pacman and ghosts at walls	Manutenção perfectiva

636c473	test: add acceptance tests for walls	Testes de aceitação (CA1 a CA9)

Figura 1 – Histórico da *branch* manutencao no repositório da dupla



2 IDENTIFICAÇÃO E CARACTERIZAÇÃO DO PROJETO

Tabela 2 – Identificação do projeto

Item	**Valor**
Projeto original	pacman-react, de Maximo Belen (mbfassnacht), 100 estrelas e 47 *forks*
URL original	https://github.com/mbfassnacht/pacman-react
Tecnologias	TypeScript 4.9, React 18 (hooks e Context API), styled-components 5, create-react-app 5 (Jest e Testing Library)
Licença	Nenhuma declarada

Objetivo. É uma versão simplificada do Pacman que roda no navegador. O jogador controla o Pacman com as setas, come as comidas do campo e evita os fantasmas. Vence ao comer todas as comidas e perde ao encostar num fantasma. Há três dificuldades (2, 3 ou 4 fantasmas, com velocidades diferentes), placar e cronômetro.

Organização e execução. O código está em src/: components/ (Scene, que monta o campo e as comidas, Pacman, Ghost, Food e Header), context/ (GameContext, o estado global), hooks/ (useInterval, o relógio do jogo, com *tick* de 100 ms), types/ e styles/. O único teste é src/App.test.tsx. O jogo roda com yarn install e yarn start, sem banco de dados, servidor ou API externa.

Histórico. São 22 *commits*. Em 2023, o *commit* 21eb137 reescreveu o projeto inteiro (105 arquivos) de JavaScript, webpack e SCSS para TypeScript, *hooks* e styled-components. Há mudanças evolutivas, corretivas, adaptativas e de documentação, mas as mensagens quase nunca explicam o motivo, e não existe nenhuma *issue* ou *pull request*.

Justificativa da escolha. O projeto é uma aplicação, e não uma biblioteca; é pequeno (cerca de 750 linhas nos componentes), mas tem lógica suficiente para análise: movimento, colisão, estado global e temporização. Usa tecnologias conhecidas pela dupla, roda localmente e tem README com instruções. Os pontos fracos foram assumidos: não há testes reais, não há *issues* e não há licença declarada. Como o uso é acadêmico e sem redistribuição, a falta de licença não impediu o trabalho, e a falta de testes obrigou a dupla a criar uma rede de segurança antes de alterar o código.

3 BASELINE

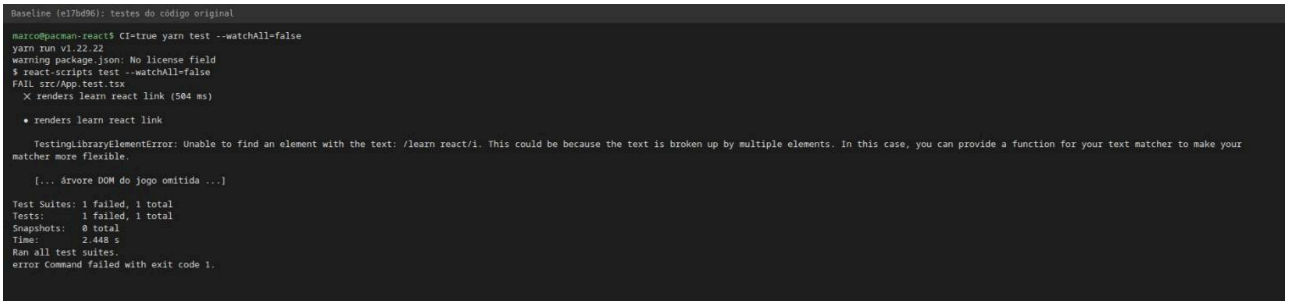
Tabela 3 – Baseline do projeto original

Informação	**Resultado**
Commit da baseline	e17bd96518a8a4ada303adc6a17b1bca17f77b4c
Ambiente	Node 20.20.2, Yarn 1.22.22, Linux
Build (yarn build)	Sucesso, com 5 avisos de lint
Quantidade de testes	1

Testes executados com sucesso	0 de 1
Execução da aplicação (yarn start)	Sucesso

O único teste é o exemplo gerado pelo create-react-app, que procura o texto "learn react" e falha porque esse texto não existe no jogo (Figura 2). Na prática, o projeto não tinha nenhum teste que verificasse o comportamento. Os 5 avisos de lint (4 de react-hooks/exhaustive-deps e 1 *import* sem uso) foram registrados como referência e conferidos a cada *commit*.

Figura 2 – Execução dos testes na baseline



Fonte: elaborado pelos autores (2026).

4 ANÁLISE DE MANUTENIBILIDADE

A análise combinou inspeção manual e análise estática (avisos do ESLint, busca de usos com grep e contagem de linhas duplicadas). O critério foi não tratar todo *code smell* como dívida técnica, mas explicar como cada problema aumenta o esforço ou o risco de mudanças futuras.

Tabela 4 – Problemas de manutenibilidade

Problema e localização	**Evidência e consequência**	**Tratamento possível**	**Prioridade e decisão**
P1 Código duplicado: Pacman.move() e Ghost.move()	Mesmo switch de 35 linhas nos dois. Toda regra de movimento nova	Extract Function + Move Function	Alta. Tratado (Refatoração 1)

	teria de ser feita duas vezes		
P2 Condicional complexa: Food.tsx e Ghost.tsx	Quatro comparações com aritmética repetida e sem nome. Ajustar a colisão é arriscado	Decompose Conditional	Média. Tratado (Refatoração 2)
P3 Estado derivado: Scene.tsx	Velocidade em useState e três if. Duas fontes da verdade e render extra	Lookup Table + Replace Derived Variable with Query	Baixa. Tratado (Refatoração 3)
P4 Contagem de comidas duplicada: Scene.tsx	Total e geração calculados de formas diferentes. Em 1920x1080, placar 496 e 465 geradas: vitória impossível	Calcular a grade em um só lugar	Alta. Achado na análise de impacto; tratado nos muros
P5 Nome enganoso: GAME_STATUS.PAUSED	Significa "não iniciado". Induz a erro ao criar novos status	Rename	Média. Não tratado
P6 Código morto: App.css, logo.svg, react-svg-inline e outros	Itens sem uso, confirmados com grep. Ruído para o leitor	Remove Dead Code	Baixa. Não tratado
P7 Acoplamento implícito: evento restart-game	Reinício por evento global no document. Dependências invisíveis	Mover o estado para o contexto	Média. Não tratado
P8 *Stale closures*: 4 avisos exhaustive-deps	*Listeners* leem o estado do primeiro render. Lógica nova neles falha em silêncio	Declarar dependências ou useRef	Média. Contornado nos muros

P9 Baixa testabilidade: projeto inteiro	Teste único falhando; lógica presa a window, Math.random e setInterval	Testes de caracterização e funções puras	Alta. Tratado em parte
P10 Números mágicos: Scene.tsx	border={20} repetido 5 vezes; props do App ignoradas	Symbolic Constant	Baixa. Não tratado

P6 e P10 são *code smells*, mas não são dívida técnica relevante aqui: não aumentam o risco da mudança planejada, e removê-los seria limpeza pela limpeza. P5 e P7 atrapalham, mas tratá-los espalharia mudanças por vários arquivos sem ganho para os muros. P1, P2 e P3 foram escolhidos para refatoração porque estavam no caminho da funcionalidade nova (movimento e colisão) e podiam ser verificados pelos testes. P4 começou como duplicação e se revelou um defeito, encontrado só na análise de impacto.

5 REFATORAÇÕES REALIZADAS

5.1 Rede de segurança de testes

Sem testes, não há como afirmar que uma refatoração preservou o comportamento. Por isso, antes de alterar o código, a dupla escreveu 19 testes de caracterização (73b60cf), que registram o comportamento atual do jogo: estado inicial, movimento e bordas, velocidades por dificuldade, cronômetro, colisão com fantasma e reinício. O tempo foi controlado com *fake timers* e o sorteio dos fantasmas foi fixado. O teste do *placeholder* passou a verificar o título "PACMAN" (de64796).

Para confirmar que esses testes detectam mudanças, foram inseridos 9 defeitos temporários no código original, como mudar a velocidade do Pacman ou desativar a colisão. Todos foram detectados. Resultado antes das refatorações: 20 testes, 20 aprovados.

5.2 Refatorações

Tabela 5 – Refatorações realizadas

Problema	**Técnica**	**Commit**
P1 Movimento duplicado em Pacman e Ghost	Extract Function + Move Function para utils/movement.ts (calculateNextPosition)	3d46458
P2 Condicionais de colisão sem nome em Food e Ghost	Decompose Conditional + Extract Variable em utils/collision.ts (isPacmanOverFood, isTouchingGhost)	308fddf
P3 Velocidade dos fantasmas como estado derivado em Scene	Replace Conditional with Lookup Table + Replace Derived Variable with Query (ghostVelocityByDifficulty)	4aeb82a

Refatoração 1. A regra "andar uma casa sem sair da tela" estava copiada nos dois personagens, e qualquer mudança nela precisaria ser feita duas vezes. Com a extração, passou a existir um único ponto de mudança, e a regra virou uma função pura, testada com 8 testes unitários (49ffcd0). Essa foi a decisão mais importante para a etapa seguinte: os muros foram adicionados como uma checagem separada, sem alterar a regra de movimento nem os testes dela. Um cuidado foi manter a leitura da posição exatamente como no original, para não mudar o comportamento do fantasma.

Refatoração 2. O leitor precisava decifrar quatro comparações com aritmética repetida para entender "o Pacman está em cima da comida?". O nome da função passou a documentar a intenção, a expressão repetida é calculada uma vez e a regra pode ser testada isoladamente, inclusive nos limites (>= contra <), com 4 testes unitários (49ffcd0).

Refatoração 3. A velocidade era guardada em estado e copiada por três if, com o valor 20 repetido em dois lugares. Com o mapa tipado como Record<Difficulty, number>, o compilador obriga a definir uma velocidade para cada dificuldade: se uma nova for criada e esquecida, o *build* falha, enquanto a cadeia de if a deixaria em 20 sem aviso. O render extra causado pelo setState também desapareceu. Embora tenha prioridade baixa, foi escolhida por estar no Scene, que seria alterado nos muros, e por ter testes que cobriam as três velocidades.

Preservação do comportamento. Após cada refatoração, foram executados os testes e o *build*. Os 20 testes escritos contra o código original continuaram passando depois das três refatorações, a suíte chegou a 32 testes com os unitários e o número de avisos de lint continuou em 5.

6 SOLICITAÇÃO DE MANUTENÇÃO PERFECTIVA

A primeira proposta, pausar a partida com a tecla P, foi considerada de escopo pequeno pela professora, que sugeriu **muros**, como no Pacman original. Ao estudar a pausa, a dupla percebeu a armadilha do status PAUSED (P5): reaproveitá-lo mostraria o seletor de dificuldade no meio da partida.

Necessidade. O campo é um retângulo vazio: Pacman e fantasmas só são limitados pelas bordas, e fugir dos fantasmas fica fácil demais. A mudança cria um conceito que o sistema não tem, o mapa, que atravessa geração de comida, movimento, pontuação, vitória e testes.

Comportamento esperado:

a) o campo exibe muros azuis finos (barras retas e formatos de "U"), alinhados à grade de 60 px e repetidos em intervalos regulares;

b) nenhuma comida é gerada dentro de um muro, e o placar mostra como total só as comidas que existem;

c) o Pacman não atravessa muros. Com a **curva antecipada**, se a direção pedida estiver bloqueada, ele continua na atual e vira quando a passagem abrir; com o **ajuste no canto**, se faltar até 30 px para alinhar com uma passagem, ele se encaixa nela;

d) os fantasmas não atravessam muros e, ao serem bloqueados, sorteiam nova direção no *tick* seguinte;

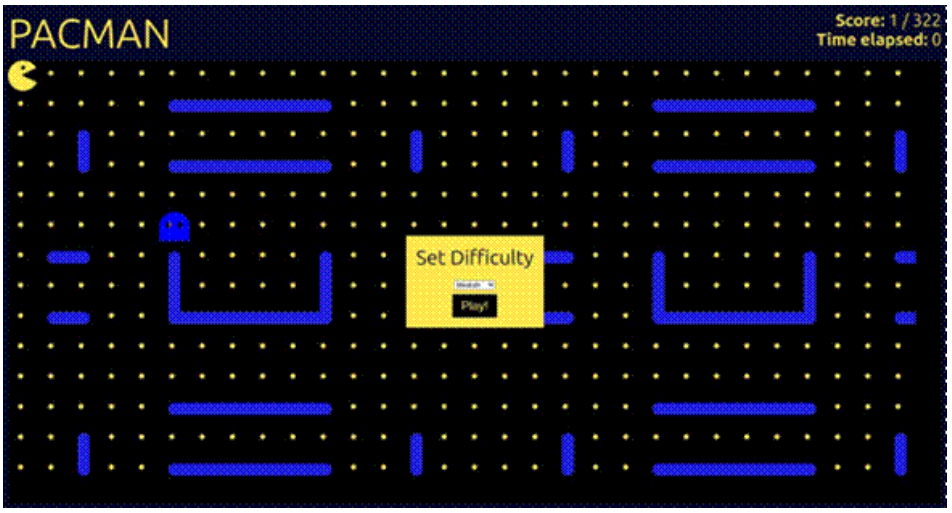
e) o jogador vence quando come todas as comidas existentes.

Regras: o desenho de 16x10 células se repete pelo campo (R1); só são desenhados muros que cabem na grade (R2); as posições iniciais nunca são muro (R3); toda comida deve ser alcançável (R4); os muros não mudam no reinício (R5); o comportamento existente é preservado (R6); nenhuma dependência nova (R7). Ficam fora do escopo um labirinto fiel ao original, fantasmas que perseguem o Pacman e editor de mapas.

Tabela 6 – Critérios de aceitação

ID	Critério
CA1	Na janela de 1024x768, 34 células de muro aparecem nas posições do mapa
CA2	Nenhuma comida ocupa um muro, e o placar mostra "1 / 126"
CA3	O Pacman para encostado no muro, sem sobrepô-lo
CA4 / CA4b	O Pacman volta a se mover ao escolher direção livre e vira na primeira passagem, mesmo com a seta apertada meio passo antes
CA5 / CA6	Fantasmas não entram em muros e, bloqueados, sorteiam nova direção no *tick* seguinte
CA7	Após "Try Again", os muros continuam nas mesmas posições
CA8	Em outros tamanhos de janela, só aparecem os muros que cabem, e o total é o de células livres
CA9	Todos os testes existentes continuam passando, mudando só o total de comidas

Figura 3 – Versão final: campo com muros



7 ANÁLISE DE IMPACTO

7.1 Impacto previsto

A análise foi registrada antes da implementação, a partir da pergunta: se colocarmos muros no campo, quais partes precisarão ser modificadas ou verificadas?

Tabela 7 – Impacto previsto

Elemento	**Tipo**	**O que muda ou deve ser conferido**	**Risco**
utils/walls.ts e walls.test.ts	Criar	Regra do mapa (isWallCell), lista de muros e sobreposição com muro (hitsWall), com testes	Baixo
components/Wall.tsx	Criar	Desenho do muro, no padrão de Food.tsx	Baixo
styles/Colors.ts	Modificar	Cor do muro	Baixo
context/GameContext.tsx	Modificar	Novo campo walls: mudança de interface do contexto	Médio
components/Scene.tsx	Modificar	Grade calculada uma vez; comida fora dos muros; total = células menos muros; desenho dos muros	Alto (R-A)
Pacman.tsx e Ghost.tsx	Modificar	Bloquear no muro; fantasma sorteia nova direção ao ser bloqueado	Médio (R-B, R-C)
utils/movement.ts	Verificar	Não deve mudar: o muro é checado à parte, e os 8 testes continuam valendo	Baixo
Food.tsx, Header.tsx, collision.ts, tipos	Verificar	Não devem mudar; a vitória depende de o total estar correto	Médio
Game.test.tsx	Modificar	Só o total esperado (160 para 136 no mapa previsto) e testes novos por critério	Médio (R-D)
Dependências e dados	Verificar	Nenhuma dependência nova; o jogo não persiste dados	Baixo

R-A, contagem inconsistente. O total do placar e as comidas geradas usavam cálculos diferentes e divergiam em algumas resoluções: em 980x768, 160 contra 150; em 1920x1080, 496 contra 465. Nesses casos, era impossível vencer. Como os muros obrigam a mexer nesse código, a dupla decidiu unificar o cálculo da grade e registrar a correção como mudança de comportamento, em vez de escondê-la. **R-B, fantasmas presos:** forçar o sorteio no *tick* seguinte. **R-C, desalinhamento:** fora das posições múltiplas de 60 px, o personagem pode não caber num corredor; registrado como limitação. **R-D, alterar testes de caracterização:** só o total muda, porque é o que a solicitação pede, e a alteração entra no mesmo *commit* da mudança. **R-E, stale closure:** ler os muros dentro de move() é seguro, ao contrário dos *listeners* (P8).

O plano previu quatro *commits* (funções do mapa; desenho e comida; bloqueio; testes de aceitação), com *build*, tsc, suíte completa e contagem de avisos após cada um. A implementação seguiu esse plano (5c3ac50, 131541e, 6e2bb9f e 636c473), e cada *commit* foi verificado isoladamente.

7.2 Impacto previsto versus observado

Tabela 8 – Impacto previsto e observado

Elemento	**Previsto**	**Observado**
walls.ts	Mapa, lista de muros e hitsWall	Criado, com uma função a mais (moveAvoidingWalls), da curva antecipada
Wall.tsx	Simples, como Food.tsx	Mais elaborado: muro fino, ligado às células vizinhas
Colors.ts, GameContext.tsx, Scene.tsx, Ghost.tsx	Modificar	Como previsto; o defeito R-A foi corrigido e virou teste
Pacman.tsx	Só bloquear no muro	**Além do previsto:** novo estado nextDirection, curva antecipada e ajuste no canto

movement.ts e demais arquivos de verificação	Não mudar	Não mudaram
Game.test.tsx	Total 160 para 136	Total 160 para **126** (mapa redesenhado, de 24 para 34 muros) e 10 testes novos

Os 9 arquivos previstos foram exatamente os alterados, e nenhum arquivo fora da previsão foi tocado. O maior desvio foi o risco R-C: tratado como limitação, ele tornava o jogo quase injogável perto dos muros e exigiu a curva antecipada. A aparência dos muros também não tinha sido considerada. Houve ainda uma pequena mudança perceptível: a virada do Pacman passou a ocorrer no próximo *tick*, e não no instante em que a seta é apertada.

Três decisões de implementação explicam por que o impacto ficou contido. Primeiro, calculateNextPosition não foi alterada: o muro é verificado depois do cálculo da próxima posição, o que preservou a Refatoração 1 e seus testes. Segundo, os muros ficaram no GameContext, como já acontecia com o total de comidas, sem alterar o tipo dos personagens. Terceiro, a curva antecipada só age quando há um muro no caminho; em campo aberto, o movimento é idêntico ao original, e por isso os testes de caracterização não precisaram mudar.

8 VERIFICAÇÃO E TESTES

Tabela 9 – Evolução da suíte de testes em relação à baseline

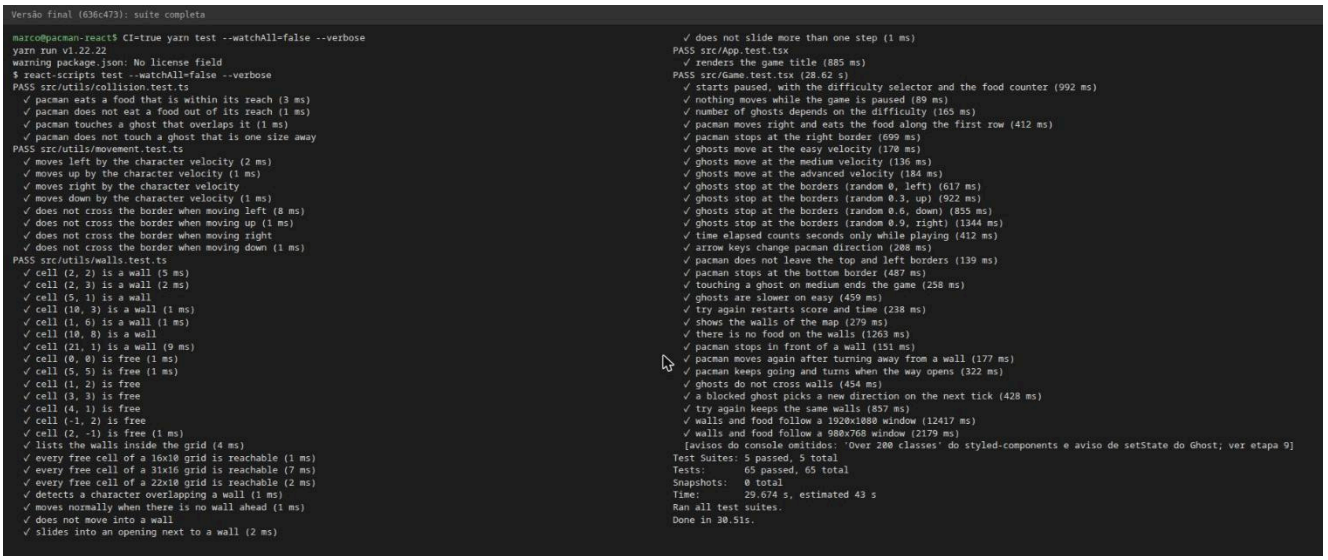
Momento	**Commit**	**Testes**	**Aprovados**	**Avisos de lint**
Baseline	e17bd96	1	0	5
Testes de caracterização	de64796	20	20	5
Após as refatorações	4aeb82a	32	32	5
Após o mapa e o desenho dos muros	131541e	51	51	5
Após o bloqueio nos muros	6e2bb9f	55	55	5
Final	636c473	65	65	5

Os 65 testes finais são 19 de caracterização, 10 de aceitação (CA1 a CA9), 1 do título, 8 de movimento, 4 de colisão e 23 das funções dos muros (Figura 4). **Regressão:** os 19 testes de caracterização continuam passando, e a única alteração neles foi o total de comidas. Nenhuma posição, tempo ou velocidade esperada mudou. Também não mudaram movement.ts, collision.ts, Food.tsx, Header.tsx, os testes unitários das refatorações, package.json e yarn.lock. O *build* manteve os mesmos 5 avisos da *baseline*.

Mutação manual. Foram inseridos 13 defeitos de propósito, um por vez, como o Pacman atravessar muros, comida gerada dentro do muro, total sem descontar os muros e mapa com um muro a menos. Todos foram detectados. Em rodadas anteriores, dois tinham passado despercebidos, o que levou a reforçar os testes com uma janela de 980x768 e com o caso de linha negativa na grade.

Verificação manual. No Chrome, o placar ficou coerente com as comidas na tela, o Pacman parou ao encostar nos muros, entrou em corredores estreitos com a seta apertada antes da hora e o "Try Again" manteve o mesmo mapa. Também foram jogadas partidas completas.

Figura 4 – Suíte completa na versão final: 65 de 65 testes aprovados



9 REFLEXÃO FINAL

A primeira dificuldade foi não ter como saber se algo quebrou. O único teste falhava, e escrever testes de caracterização deu mais trabalho do que imaginávamos, porque foi preciso controlar o tempo, o sorteio e o tamanho da

janela. Também aprendemos que teste verde não basta: só a mutação manual mostrou que alguns testes deixavam defeitos passar.

O código esconde decisões que só aparecem lendo com atenção: PAUSED significava "não iniciado", o reinício dependia de eventos globais e alguns *listeners* liam estado antigo. Sem documentação nem *issues*, o histórico de *commits* era a única fonte.

O ambiente também é do terceiro: usamos as mesmas versões de Yarn e Prettier do projeto e acompanhamos os avisos da *baseline* a cada *commit*, para ter certeza de que não estávamos piorando o software. Separar o trabalho em *commits* pequenos, cada um compilando e passando nos testes, foi o que permitiu rastrear cada decisão até o código.